Système d'exploitation : TP1 (fork et wait)

janvier 2026 — Pierre Rousselin

Exercice 1 : `fork` et `wait`

1. Télécharger sur l'ENT le fichier `exemple_wait_tp1.c`. Le compiler et l'exécuter plusieurs fois. Que se passe-t-il ?
2. Combien de processus sont créés lorsque vous éxécutez le programme ?
3. À quoi sert l'appel système `wait` ? Que retourne-t-il ? (Pour l'argument pointeur, on verra plus tard.)
4. Tous les `wait` de ce programme sont-ils utiles ?
5. Commenter les appels systèmes `wait` (et les affichages correspondants) et observer la différence.
6. Avec quelles instructions un processus peut-il attendre la mort de tous ses enfants, même sans en connaître le nombre ?

--- * ---

Correction de l'exercice 1 :

1. Les résultats sont aléatoires, mais les deux enfants meurent toujours avant le parent et la valeur de retour du dernier `wait` est toujours `-1`.
2. Le processus créé par le shell pour exécuter le programme, plus les deux enfants de ce processus, ça fait 3.
3. `wait` est bloquant, si un enfant a déjà terminé il retourne tout de suite avec comme valeur de retour le `pid` de celui-ci, sinon le processus est endormi et réveillé à la mort d'un enfant.
   S'il n'y a plus d'enfant à attendre, `wait` retourne `-1` (sans bloquer).
4. Le troisième ne sert à rien, il n'y a plus d'enfant à attendre.
5. Il n'y a plus aucune synchronisation et rien n'empêche le parent de mourir avant ses enfants.
6. ```
   while (wait(NULL) != -1)
           ;
   ```

--- * ---

Exercice 2 : Mousse au chocolat avec `fork` et `wait`

1. Télécharger sur l'ENT le fichier `mousse_au_chocolat.c`. Le compiler et l'exécuter avec chronomètre de la façon suivante :
   ```
   $ gcc -Wall mousse_au_chocolat.c -o mousse
   $ time ./mousse
   ```
   Le champ `real` correspond au temps entre la fin du processus et son début. Les champs `user` et `sys` correspondent aux temps de calcul du processeur respectivement en mode utilisateur et privilégié (noyau). Pourquoi sont-ils sensiblement plus faibles que `real` ?
2. À vous de jouer : en utilisant `fork` et `wait`, réduire le plus possible la durée `real` en respectant les dépendances entre les tâches (par exemple les œufs doivent être séparés en blancs et jaunes avant que les blancs puissent être battus en neige, voir exercice 1 du TD 1).

--- * ---

Correction de l'exercice 2 : Pour la question 1 : c'est parce que presque tout le temps est passé dans `usleep`, le processus est endormi donc ne prend pas de temps de calcul au processeur.

Pour la question 2 : première solution optimale dans ce cas, mais dans la vraie vie on ne sait pas à l'avance la durée des tâches.
   ```

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

void casser_chocolat(void);
void faire_fondre_chocolat(void);
void separer_blanc_jaune(void);
void melanger_jaunes_choco(void);
void battre_blancs_neige(void);
void incorporer_blancs_neige(void);

int main(void)
{
        int i;

        switch (fork()) {
        case -1:
                perror("fork");
                return 1;
        case 0: /* batteur d'oeufs */
                for (i = 0; i < 6; ++i) {
                        switch (fork()) {
                        case -1:
                                perror("fork");
                                return 1;
                        case 0:
                                separer_blanc_jaune();
                                return 0;
                        }
                }
                while (wait(NULL) != -1)
                        ;
                battre_blancs_neige();
                return 0;
        default:
                switch (fork()) {
                case -1:
                        perror("fork");
                        return 1;
                case 0: /*fondeur de chocolat */
                        casser_chocolat();
                        faire_fondre_chocolat();
                        return 0;
                }
        }
        /* On attend tout le monde et on termine la recette. */
        while (wait(NULL) != -1)
                ;

        melanger_jaunes_choco();
        incorporer_blancs_neige();

        printf("PID %d : la mousse est terminée !\n", getpid());
        return 0;
}

void casser_chocolat(void)
```

```
{
        printf("PID %d : je casse le chocolat.\n", getpid());
        usleep(200000);
        printf("PID %d : le chocolat est cassé !\n", getpid());
}

void faire_fondre_chocolat(void)
{
        printf("PID %d : je fais fondre le chocolat.\n", getpid());
        usleep(1000000);
        printf("PID %d : le chocolat est fondu !\n", getpid());
}

void separer_blanc_jaune(void)
{
        printf("PID %d : je sépare le blanc d'un jaune.\n", getpid());
        usleep(200000);
        printf("PID %d : un blanc est séparé d'un jaune !\n", getpid());
}

void melanger_jaunes_choco(void)
{
        printf("PID %d : je mélange les jaunes et le chocolat.\n", getpid());
        usleep(200000);
        printf("PID %d : le chocolat est mélangé avec les jaunes !\n", getpid());
}

void battre_blancs_neige(void)
{
        printf("PID %d : je bas les blancs en neige.\n", getpid());
        usleep(600000);
        printf("PID %d : les blancs sont battus en neige !\n", getpid());
}

void incorporer_blancs_neige(void)
{
        printf("PID %d : j'incorpore les blancs en neige.\n", getpid());
        usleep(1000000);
        printf("PID %d : les blancs sont incorporés !\n", getpid());
}
```

Deuxième correction sans tricher (merci Antoine) avec utilisation de la valeur de retour de `wait` pour savoir qui finit avant.

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

void casser_chocolat(void);
void faire_fondre_chocolat(void);
void separer_blanc_jaune(void);
void melanger_jaunes_choco(void);
void battre_blancs_neige(void);
void incorporer_blancs_neige(void);

int main(void)
{
        int i;
```

```
pid_t oeufs, choco, p;

switch ((oeufs = fork())) {
case -1:
        perror("fork");
        return 1;
case 0: /* batteur d'oeufs */
        for (i = 0; i < 5; ++i) {
                switch (fork()) {
                case -1:
                        perror("fork");
                        return 1;
                case 0:
                        separer_blanc_jaune();
                        return 0;
                }
        }
        separer_blanc_jaune();
        while (wait(NULL) != -1)
                ;
        return 0;
default:
        switch ((choco = fork())) {
        case -1:
                perror("fork");
                return 1;
        case 0: /*fondeur de chocolat */
                casser_chocolat();
                faire_fondre_chocolat();
                return 0;
        }
}

if ((p = wait(NULL)) == oeufs) {
        /* oeufs séparés avant que le chocolat soit fondu */
        switch (fork()) {
        case -1:
                perror("fork");
                return 1;
        case 0:
                battre_blancs_neige();
                return 0;
        }
        if ((p = wait(NULL)) == choco) {
                /* chocolat fondu avant que les oeufs soient montés */
                switch (fork()) {
                case -1:
                        perror("fork");
                        return 1;
                case 0:
                        melanger_jaunes_choco();
                        return 0;
                }
        } else {
                /* il faut de toute façon attendre que le chocolat soit
                 * fondu */
                wait(NULL);
```

```
                        melanger_jaunes_choco();
                }
        } else {
                /* chocolat fondu avant que les oeufs soient séparés
                 * on ne peut rien faire d'autre, il faut attendre. */
                wait(NULL);
                switch (fork()) {
                case -1:
                        perror("fork");
                        return 1;
                case 0:
                        melanger_jaunes_choco();
                        return 0;
                }
                switch (fork()) {
                case -1:
                        perror("fork");
                        return 1;
                case 0:
                        battre_blancs_neige();
                        return 0;
                }
                /* On attend tout le monde pour pouvoir finir la recette */
                while (wait(NULL) != -1)
                        ;
        }
        incorporer_blancs_neige();

        printf("PID %d : la mousse est terminée !\n", getpid());
        return 0;
}

void casser_chocolat(void)
{
        printf("PID %d : je casse le chocolat.\n", getpid());
        usleep(200000);
        printf("PID %d : le chocolat est cassé !\n", getpid());
}

void faire_fondre_chocolat(void)
{
        printf("PID %d : je fais fondre le chocolat.\n", getpid());
        usleep(1000000);
        printf("PID %d : le chocolat est fondu !\n", getpid());
}

void separer_blanc_jaune(void)
{
        printf("PID %d : je sépare le blanc d'un jaune.\n", getpid());
        usleep(200000);
        printf("PID %d : un blanc est séparé d'un jaune !\n", getpid());
}

void melanger_jaunes_choco(void)
{
        printf("PID %d : je mélange les jaunes et le chocolat.\n", getpid());
        usleep(200000);
```

```
        printf("PID %d : le chocolat est mélangé avec les jaunes !\n", getpid());
}

void battre_blancs_neige(void)
{
        printf("PID %d : je bas les blancs en neige.\n", getpid());
        usleep(600000);
        printf("PID %d : les blancs sont battus en neige !\n", getpid());
}

void incorporer_blancs_neige(void)
{
        printf("PID %d : j'incorpore les blancs en neige.\n", getpid());
        usleep(1000000);
        printf("PID %d : les blancs sont incorporés !\n", getpid());
}
```

--- * ---